Gitea ile Python Docker Image Pipeline

Adım adım uygulama rehberi • Beyza için • 3 Kasım 2026

Hedef

Python kodunu Gitea repository’sine gönderdiğinde build otomatik başlayacak, Dockerfile kullanılarak image oluşturulacak ve Gitea Container Registry’ye yüklenecek. Son olarak image registry’den çekilerek çalıştırılacak.

Bileşenler ve görevleri

Bileşen	Görevi
Gitea repository	Python kodu, Dockerfile ve workflow dosyasını saklar.
Gitea Actions	Push olayında yapılacak işi tanımlar.
Gitea Runner	Workflow komutlarını çalıştırır.
Dockerfile	Image oluşturma tarifidir.
Docker Engine	Image oluşturur ve registry’ye gönderir.
Gitea Container Registry	Oluşan image’ları saklar.

Case’in kapsamı

Gitea Container Registry, Docker image’larını saklayabilen bir registry’dir. Ayrı bir Docker Hub veya şirket registry’si istenmediyse ikinci registry gerekmez. İkinci hedef isteniyorsa ona ayrıca login, tag ve push adımları eklenir.

Bu rehber image yayınlamayı kapsar. Sunucuya otomatik deployment, bu case’e sonradan eklenebilecek ayrı bir aşamadır.

1. Ortamı hazırla

Bu rehber, çalışan bir Gitea sunucusu ve Docker kurulu bir Ubuntu makinesi olduğunu varsayar. Gitea sıfırdan sunucu kurulumu burada kapsam dışındadır. Ubuntu sanal makinen runner makinesi olabilir.

Örnek değer	Kendi ortamındaki karşılığı
https://gitea.example.com	Çalışan Gitea sunucunun HTTPS adresi
beyza	Kullanıcı veya organizasyon adı
python-demo	Repository ve örnek image adı
main	Pipeline'ı tetikleyecek dal

gitea.example.com örnek adrestir; gerçek sunucu değildir. Komutlarda bu adresi, kullanıcı/organizasyon adını ve gerekiyorsa portu değiştir. Image yollarını küçük harflerle kullan.

Ubuntu üzerinde kontrol

```
docker --version
docker info
git --version
node --version
```

Runner kullanıcısı docker info komutunu sudo olmadan çalıştırabilmeli. Git ve checkout@v4 için Node.js 20 kurulu olmalı. Bu rehberin runner'ı host modunda çalışır; Ubuntu'daki programları kullanır.

Runner makinesi Gitea'ya, checkout action için GitHub'a ve Python temel image'ını indirmek için Docker Hub'a erişebilmeli. Kapalı şirket ağında kurumun action aynası ve image registry'si gerekebilir.

HTTPS sertifikası Ubuntu ve Docker tarafından güvenilir olmalı. HTTP veya özel sertifikalı bir ortam kullanıyorsan önce bu ortamın registry erişimini yapılandır; HTTPS örneğini aynen kullanma.

Repository oluştur ve klonla

Gitea'da python-demo repository'sini oluştur. İlk klonlama için README ile başlatabilirsin. Private repository'de Git işlemleri için ayrıca kimlik doğrulama gerekebilir.

```
git clone https://gitea.example.com/beyza/python-demo.git
cd python-demo
mkdir -p .gitea/workflows
```

Dosyalar: app.py, Dockerfile, .dockerignore ve .gitea/workflows/build.yml. Sonraki sayfalardaki içerikleri bu dosyalara kaydet.

2. Python uygulaması ve Dockerfile

app.py

```
from http.server import BaseHTTPRequestHandler, HTTPServer


class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        body = "Merhaba Beyza! Gitea pipeline calisiyor.\n".encode()

        self.send_response(200)
        self.send_header("Content-Type", "text/plain; charset=utf-8")
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)


if __name__ == "__main__":
    HTTPServer(("0.0.0.0", 8000), Handler).serve_forever()
```

Uygulama 8000 portunda HTTP isteği bekler. Python'ın standart kütüphanesini kullandığı için requirements.txt ve pip kurulumu gerekmez. Bu, eğitim amaçlı basit HTTP sunucusudur.

Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

COPY app.py .

USER 10001

EXPOSE 8000

CMD ["python", "app.py"]
```

FROM temel image'ı seçer. WORKDIR çalışma dizinini ayarlar. COPY uygulamayı image'a ekler. USER uygulamayı root yerine sayısal bir kullanıcıyla çalıştırır. CMD container başlatıldığında Python'ı çalıştırır.

EXPOSE 8000 yalnızca portu belgeler; Ubuntu'da port açmaz. Hosttan erişim için docker run sırasında -p kullanılır.

3. Dockerfile’ı elle doğrula

.dockerignore

```
.git
.gitea
__pycache__
*.pyc
.venv
.env
```

Bu dosya, gereksiz dosyaların ve .env gibi yerel ayarların Docker build bağlamına gönderilmesini önler.

Image oluştur ve container başlat

Aşağıdaki komutları Dockerfile’ın bulunduğu python-demo klasöründe çalıştır:

```
docker build -t python-demo:local .
docker run -d --name python-demo-test \
  -p 8000:8000 python-demo:local
curl http://localhost:8000
```

Beklenen cevap:

```
Merhaba Beyza! Gitea pipeline calisiyor.
```

Build sonundaki nokta, mevcut klasörü build bağlamı olarak seçer. -t image’a isim ve etiket verir. -p 8000:8000, Ubuntu’nun 8000 portunu container’ın 8000 portuna bağlar.

Testi bitir

```
docker rm -f python-demo-test
```

Bu komut yalnızca yukarıda oluşturduğun test container’ını kaldırır. Image makinede kalır.

Bu aşamada neyi doğruladın?

Python uygulaması çalışıyor, Dockerfile doğru ve Docker Engine image oluşturabiliyor. Pipeline aynı build işlemini runner üzerinden otomatik yapacak.

Container başlamazsa docker logs python-demo-test komutuyla logları incele. Port kullanımda hatası varsa başka bir host portu seçebilirsin: -p 8001:8000. Bu durumda curl http://localhost:8001 kullan.

4. Gitea Runner’ı kaydet

Repository ayarlarında Actions özelliğini etkinleştir. Actions sekmesi veya runner ayarları yoksa Gitea yöneticisinin sunucu düzeyinde Actions’ı etkinleştirmesi gerekebilir.

Repository → Settings → Actions → Runners bölümünden kayıt token’ı al. Menü adları sürüme göre değişebilir.

Runner binary’sini indir

Resmî adres: https://docs.gitea.com/runner/1/ . Buradaki release bağlantısından Gitea sürümünle uyumlu kararlı binary’yi indir. uname -m ile Ubuntu mimarisini kontrol et: x86_64 için amd64, aarch64 için arm64 seçilir. Eski sürümlerde runner act_runner adıyla bulunabilir.

```
mkdir -p ~/gitea-runner
```

İndirdiğin binary’yi bu klasöre koy ve adını runner yap. Bu klasör uygulama repository’sinden ayrı olmalıdır.

```
cd ~/gitea-runner
chmod +x runner
./runner --version
./runner register
```

Kayıt alanı	Gireceğin değer
Instance URL	https://gitea.example.com/
Registration token	Repository’den aldığın kayıt token’ı
Runner name	beyza-runner
Labels	docker-build:host

docker-build:host, docker-build etiketli işleri doğrudan Ubuntu üzerinde çalıştırır. Workflow’da runs-on: docker-build kullanılacak. Runner’ı container içinde başlatırsan host ifadesi o container’ı anlatır; bu rehber binary’yi Ubuntu üzerinde çalıştırır.

Runner’ı başlat

```
./runner daemon
```

Bu terminal ilk denemede açık kalsın. Gitea’da runner çevrimiçi görünmeli. Terminal kapanırsa runner durur; sürekli kullanımda systemd servisi kurulabilir. Kayıtta oluşan .runner dosyasını koru, repository’ye ekleme.

Host modu işleri Ubuntu’da doğrudan çalıştırır. Bu eğitim runner’ını yalnızca güvendiğin repository için kullan; Docker erişimi makine üzerinde geniş yetki sağlar.

5. Registry yetkisini tanımla

Kişisel erişim token’ı oluştur

Gitea → Kullanıcı ayarları → Applications / Access Tokens bölümünden bir token oluştur. Paket yazma yetkisi olan write:package seç. Token sahibi, hedef kullanıcı veya organizasyon alanına paket yükleyebilmelidir.

Token yalnızca ilk oluşturulduğunda gösterilebilir. Güvenli şekilde sakla; kod dosyalarına veya Git geçmişine ekleme.

Repository secrets ekle

Repository → Settings → Actions → Secrets bölümüne aşağıdaki iki değeri ekle:

Secret adı	Değeri
REGISTRY_USERNAME	beyza veya token sahibinin kullanıcı adı
REGISTRY_TOKEN	write:package yetkili erişim token’ı

İki token’ın farkı

Token	Görevi
Runner kayıt token’ı	Runner’ı Gitea’ya kaydeder.
Registry erişim token’ı	Image’ı registry’ye yüklemeye izin verir.

Registry ve image adresi

```
Registry: gitea.example.com
Image:    gitea.example.com/beyza/python-demo
Tag:      latest
```

Docker login ve image adresinde https:// kullanılmaz. Gitea özel portta sunuluyorsa portu dahil et; örneğin gitea.example.com:3000. Bu adres, runner makinesinden erişilebilir olmalı.

Image sahibi bir organizasyonsa IMAGE yolunda organizasyon adı, REGISTRY_USERNAME secret’ında ise o organizasyona yazabilen token sahibinin kullanıcı adı kullanılır.

Registry’ye image yüklemek için ayrı bir registry container’ı açman gerekmez; bu örnek Gitea’nın yerleşik Container Registry özelliğini kullanır. Yönetici paket registry’sini kapattıysa etkinleştirmelidir.

6. Workflow dosyası

Aşağıdaki içeriği repository’de .gitea/workflows/build.yml dosyasına kaydet. YAML girintileri boşluk olmalıdır. REGISTRY ve IMAGE değerlerini kendi ortamına göre değiştir.

```
name: Python Docker Build

on:
  push:
    branches:
      - main

jobs:
  build-and-push:
    runs-on: docker-build

    env:
      REGISTRY: gitea.example.com
      IMAGE: gitea.example.com/beyza/python-demo
      DOCKER_CONFIG: ${{ runner.temp }}/docker-auth

    steps:
      - name: Repository kodunu indir
        uses: https://github.com/actions/checkout@v4

      - name: Docker erisimini kontrol et
        run: docker info

      - name: Registry oturumu ac
        env:
          REGISTRY_USERNAME: ${{ secrets.REGISTRY_USERNAME }}
          REGISTRY_TOKEN: ${{ secrets.REGISTRY_TOKEN }}
        run: |
          mkdir -p "$DOCKER_CONFIG"
          printf '%s' "$REGISTRY_TOKEN" |
            docker login "$REGISTRY" \
              --username "$REGISTRY_USERNAME" \
              --password-stdin

      - name: Image olustur
        env:
          COMMIT_SHA: ${{ gitea.sha }}
        run: |
          docker build \
            --tag "$IMAGE:$COMMIT_SHA" \
            --tag "$IMAGE:latest" \
            .

      - name: Image registryye yukle
        env:
          COMMIT_SHA: ${{ gitea.sha }}
        run: |
          docker push "$IMAGE:$COMMIT_SHA"
          docker push "$IMAGE:latest"

      - name: Registry oturumunu temizle
        if: ${{ always() }}
        run: |
          docker logout "$REGISTRY" || true
          rm -rf "$DOCKER_CONFIG"
```

7. Push yap ve sonucu doğrula

Workflow yalnızca main dalına push geldiğinde çalışır. gitea.sha commit kimliğidir; aynı image hem bu kimlikle hem latest etiketiyle yüklenir. latest, yüklenen son sürümü işaret eden değişken etikettir.

DOCKER_CONFIG geçici klasörü, pipeline'ın Docker giriş bilgisini kullanıcı hesabının kalıcı Docker ayarlarından ayırır. always() adımı önceki adımlardan biri başarısız olsa da temizliği çalıştırır.

Kodları gönder

```
git add .
git commit -m "Python ve Docker build pipeline ekle"
git branch -M main
git push -u origin main
```

Bu komutlar uygulama repository'sinde çalıştırılır; runner klasöründe değil. Gitea Actions sekmesinde kod indirme, Docker kontrolü, login, build ve push adımlarını izle. Başarılı image kullanıcı veya organizasyonun Packages bölümünde görünür.

Registry'den çek ve çalıştır

```
docker login gitea.example.com --username beyza
```

Parola sorulduğunda registry erişim token'ını gir. Sonra:

```
docker pull gitea.example.com/beyza/python-demo:latest

docker run -d \
  --name python-demo \
  -p 8000:8000 \
  gitea.example.com/beyza/python-demo:latest

curl http://localhost:8000
```

Beklenen cevap: Merhaba Beyza! Gitea pipeline calisiyor. Aynı isimde bir container zaten varsa önce durumunu kontrol et; yeniden denemede farklı bir container adı ve boş host portu kullanabilirsin.

Otomasyonu gerçekten test et

app.py içindeki mesajı değiştir, commit ve push yap. Yeni Actions çalışmasını ve yeni commit etiketiyle yüklenen image'ı kontrol et. Çalışan container kendiliğinden güncellenmez; yeni image ile yeniden oluşturulmalıdır.

Tamamlanma ölçütleri: push ile job başladı; build başarılı; registry'de image ve etiketler göründü; registry'den çekilen image ile container çalıştı ve HTTP yanıtı alındı.

8. Hata çözümü ve kaynaklar

Belirti	Kontrol / çözüm
Job bekliyor	Runner çevrimiçi mi? runs-on etiketi docker-build ile eşleşiyor mu?
Workflow başlamıyor	Actions etkin mi? Dosya .gitea/workflows altında mı? Push main dalına mı yapıldı?
docker: command not found	Runner'ın Ubuntu ortamında Docker CLI kurulu ve PATH içinde mi?
Docker permission denied	Runner kullanıcısı sudo olmadan docker info çalıştırabiliyor mu?
unauthorized / denied	Secret adlarını, token'ın write:package yetkisini ve image sahibine yazma iznini kontrol et.
HTTP response to HTTPS client	Registry HTTP çalışıyordur. HTTPS yapılandır veya lab için Docker daemon'ın insecure registry ayarını yöneticiyle düzenle.
x509 sertifika hatası	Gitea sertifikasının CA zincirini Ubuntu ve Docker için güvenilir hale getir.
Checkout indirilemiyor	GitHub erişimini, DNS'i ve proxy ayarlarını kontrol et. Node.js 20 ve Git kurulu olmalı.
connection refused	Gitea URL, port, DNS ve firewall kontrolü yap. Container içindeki localhost o container'ı gösterir.
Port already allocated	8000 host portu dolu; -p 8001:8000 kullan ve 8001 üzerinden test et.

Resmî kaynaklar

Gitea Runner kurulum ve çalışma modları:
https://docs.gitea.com/runner/1/

Runner label yapısı:
https://docs.gitea.com/runner/labels/

Gitea Container Registry, image isimlendirme ve push/pull:
https://docs.gitea.com/usage/packages/container/

Erişim token'ları ve yetkiler:
https://docs.gitea.com/1.26/development/api-usage

Not: Komutlar örnek ortam için hazırlanmıştır; kullanıcının makinesinde çalıştırılmamıştır. Gitea ve runner sürümüne göre menü ve binary adları değişebilir.